EMBEDDED QA · DEMO DAY

SENTRY SWM-2

Hardware-in-the-Loop Test Automation Prototype

From manual firmware testing to automated physical actions and camera-based verification

ESP32-S3 • Python / pytest • USB Relay • Raspberry Pi Camera

The problem: testing real embedded behavior

Manual testing validated the firmware. The next question was:

Can we automate physical interactions and verify the observable result?

MANUAL QA

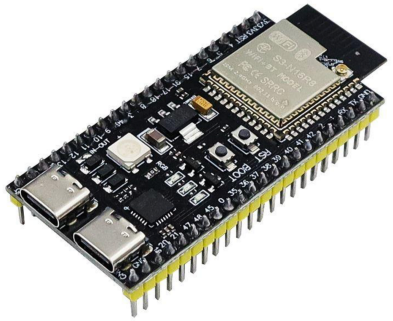
Firmware v2.1.0 → v2.3.0
OTA, negative and regression testing

HIL PROTOTYPE

Automated button presses, UART checks,
and camera-based LED verification

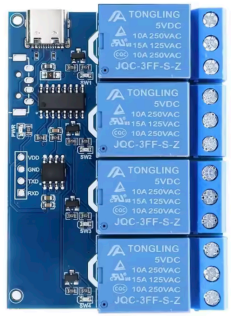
HIL TEST BENCH / HARDWARE COMPONENTS

SENTRY SWM-2 • Seven physical test-bench components



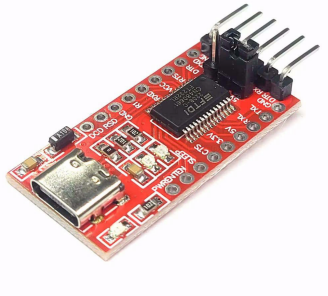
ESP32-S3 N16R8

Device Under Test (DUT)



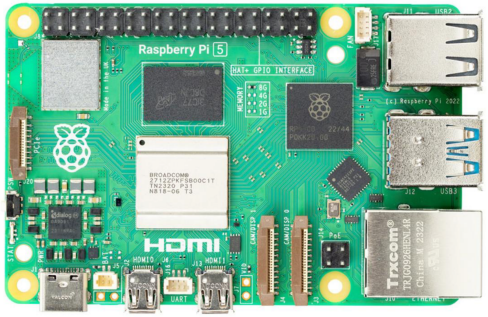
USB 4-Channel Relay

Automated K1–K4 button presses



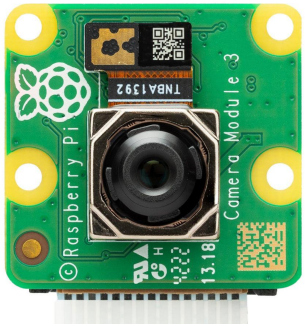
FT232RL USB–UART

Serial commands and logs



Raspberry Pi 5

Image processing

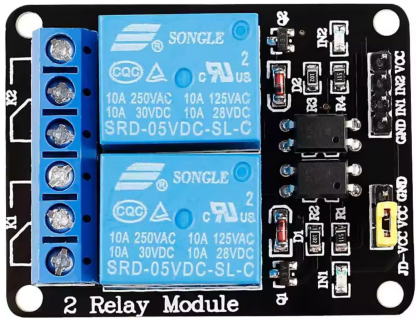


Camera Module 3

Camera-based LED observation

HC-SR04 Sensor

Ultrasonic distance measurement



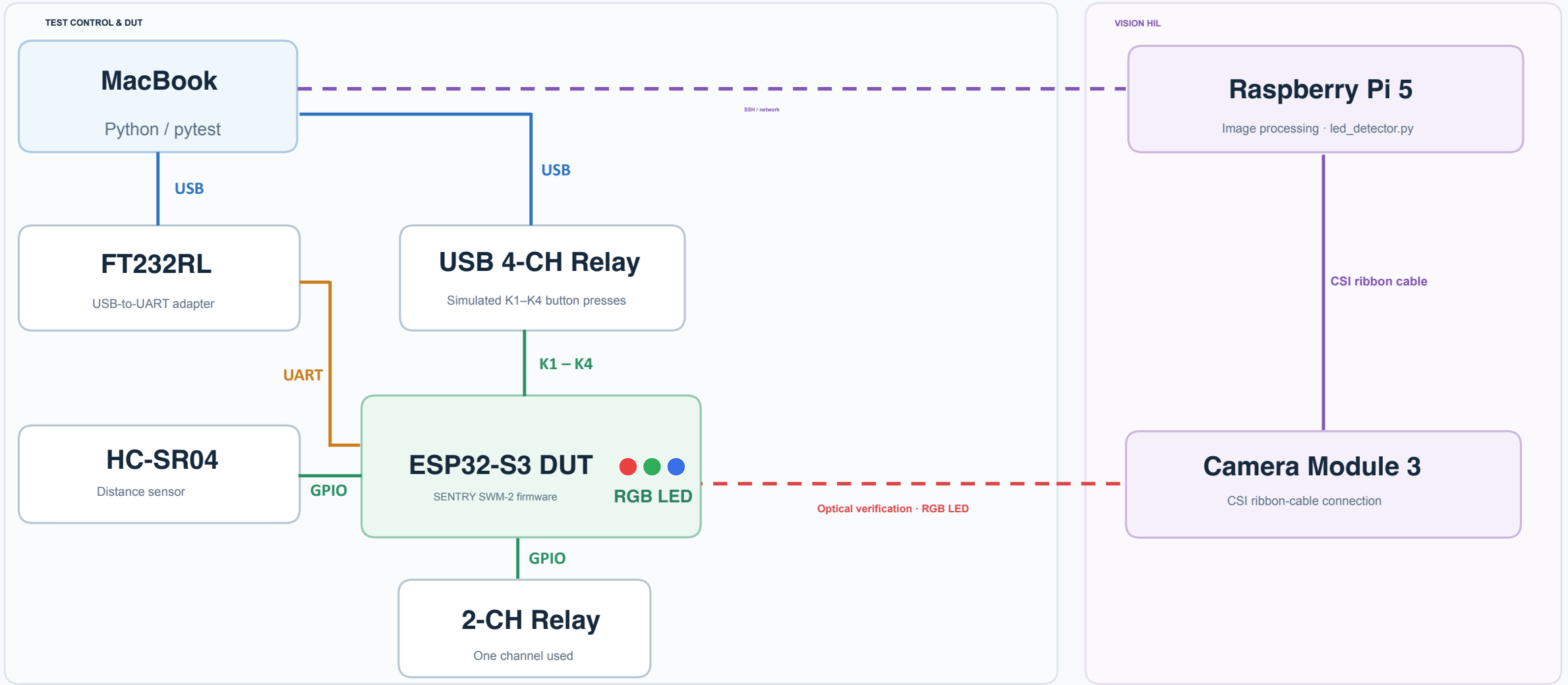
2-Channel Relay

DUT output (CH1 used)

Control: Python / pytest on MacBook • Observation: UART + Raspberry Pi camera

Test Stand Architecture

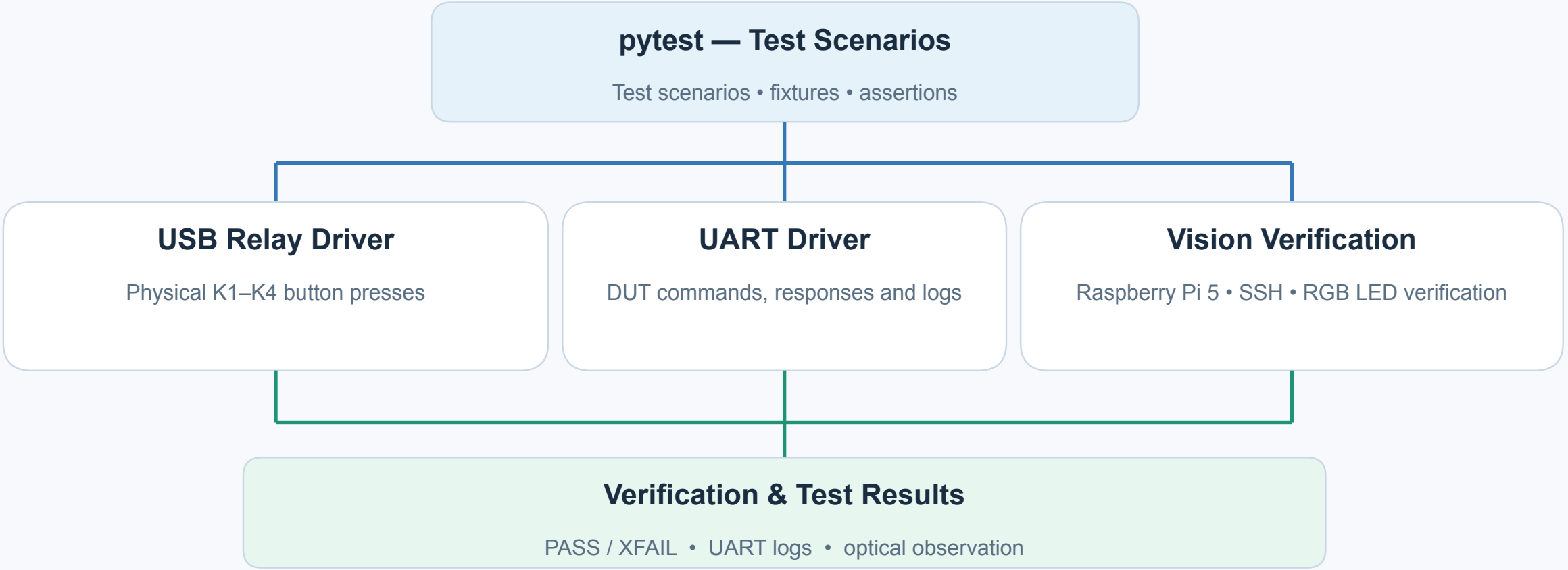
SENTRY SWM-2 | Automated Hardware-in-the-Loop Testing



Solid: USB / UART / GPIO / CSI Dashed: SSH / optical observation

Automation Framework

Python / pytest • separation of scenarios, hardware control and assertions



Case study: K1 lamp + computer vision

- USB relay physically presses the K1 button
- DUT changes lamp state / LED indication
- Raspberry Pi camera captures the LED
- Image analysis verifies the expected color
- UART provides a second observation channel

VISION ROI

40 × 40 px

RED • GREEN

Recalibration after
test-bench repositioning

VISION HIL: 2 TARGETED TESTS PASSED

Case study: physical relay control

Software checks alone cannot prove that a physical output changed state.

ACTION

USB relay presses a real DUT button

OBSERVATION

UART response + physical behavior

ASSERTION

Expected state versus actual state

Repeatable end-to-end verification: control → firmware → hardware

Defects: expected failures are still defects

BUG-CIC-01

Alert acknowledgment by ID

Expected: acknowledge the requested alert ID.
Actual: a different alert is acknowledged, although the console reports success.

BUG-RL-01

Relay pulse boundary value

Expected: reject invalid pulse value 99.
Actual: the valve physically actuates instead of returning E603.

XFAIL = KNOWN, UNRESOLVED FIRMWARE DEFECT

Regression: repeatability demonstrated

TWO FINAL FULL FRAMEWORK RUNS

RUN 1

PASS	XFAIL	FAIL
8	2	0

151.59 s

RUN 2

PASS	XFAIL	FAIL
8	2	0

151.55 s

10 tests collected • 0 unexpected failures • 2 known defects remain

Lessons learned & live demonstration

- HIL makes physical interactions repeatable
- Vision verifies the device’s actual indication
- UART and camera provide complementary evidence
- Stable camera positioning and ROI calibration matter
- QA release recommendation for v2.3.0 beta: NO-GO

LIVE DEMO → pytest → USB relay → ESP32-S3 → camera / UART → assertion